

AUTONOMOUS SYSTEMS AND NAVIGATION

UNIT 3

Outline



Autonomous robot behavior



SLAM (Simultaneous Localization and Mapping)



Navigation in unknown environments



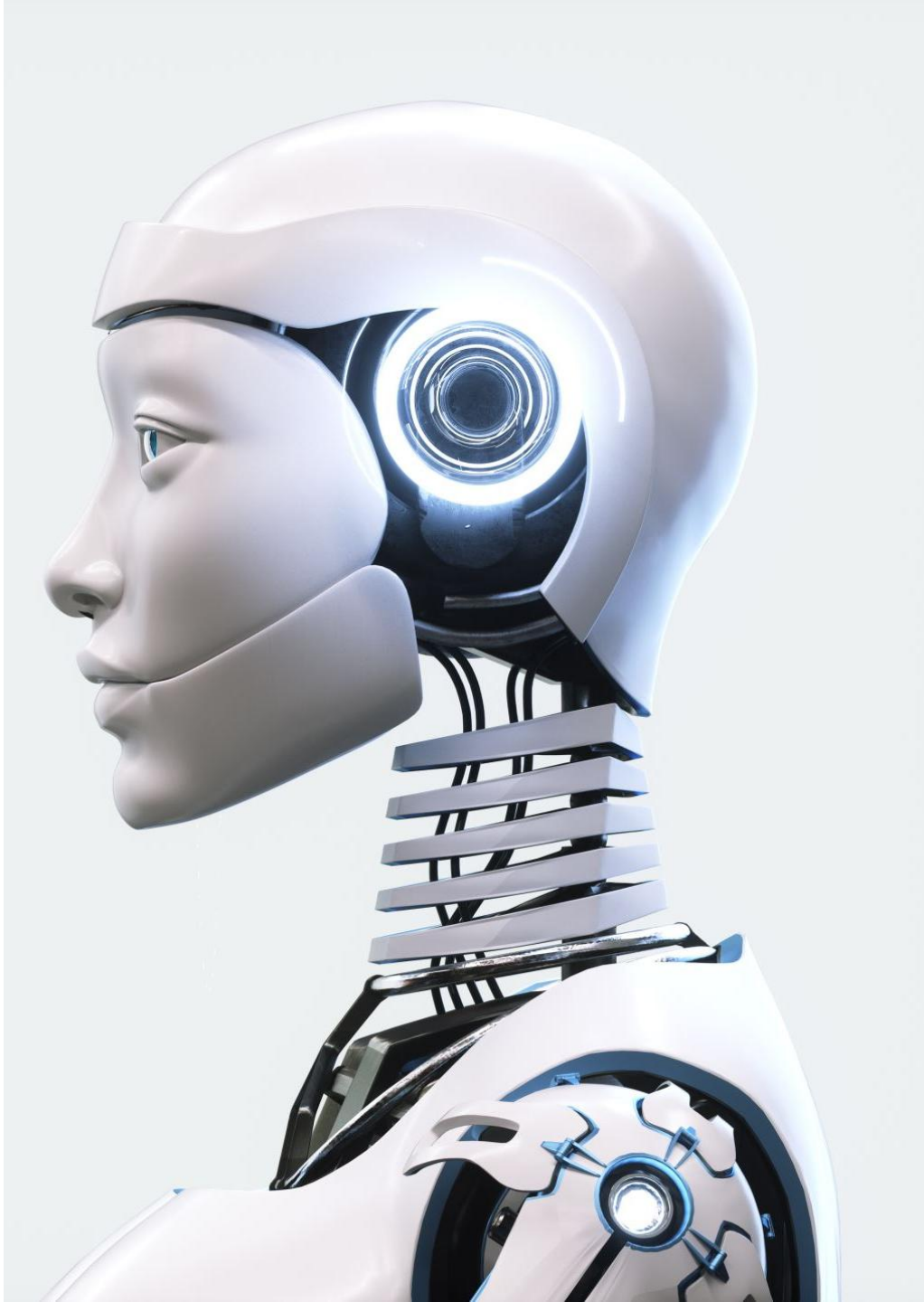
Multi-robot systems and swarm robotics



Robot programming and applications: Robot Operating System (ROS) - Simulation, Working, Applications, and Benefits.

Autonomous robot behavior

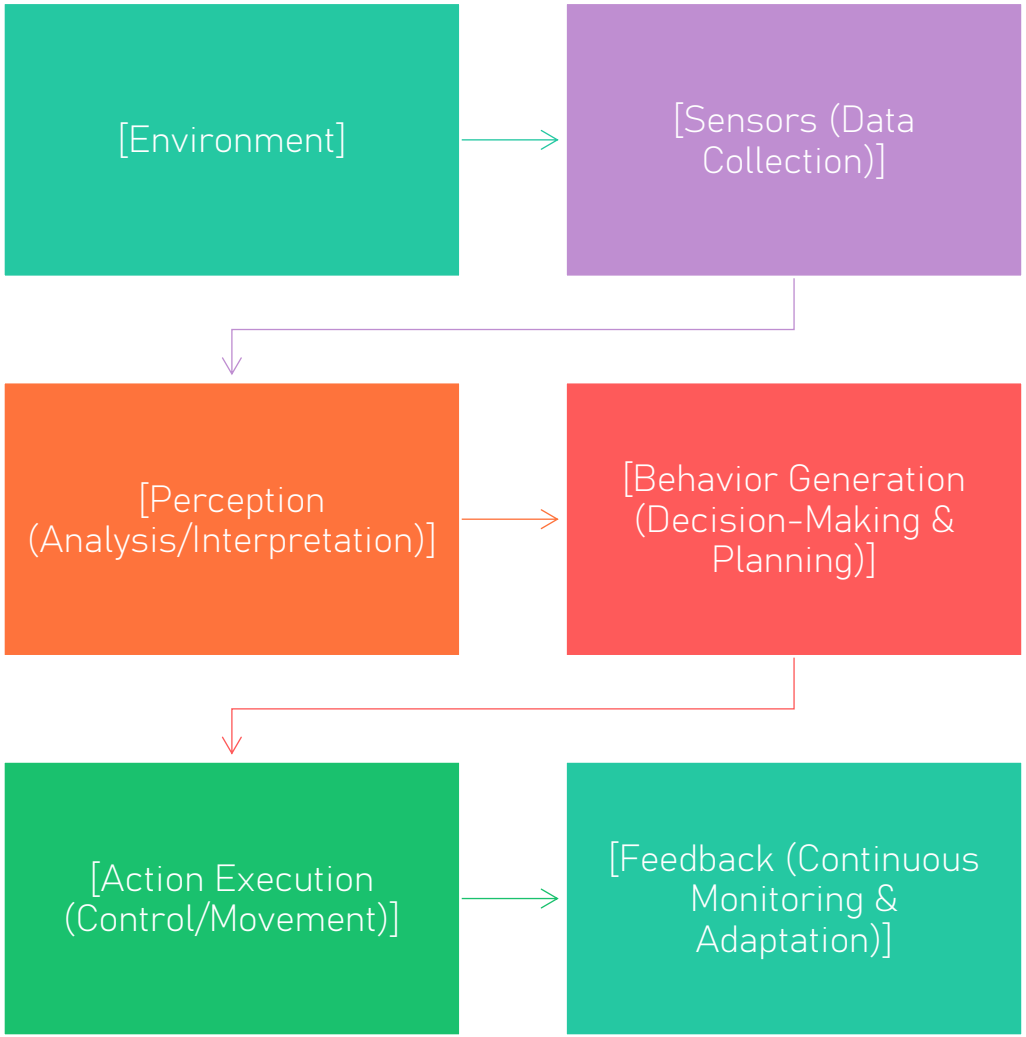
- Autonomous robot behavior is the capability of a robot to perform tasks, make decisions, and operate independently in its environment with minimal or no human intervention.
- These behaviors are achieved through the integration of advanced sensing, perception, planning, and control systems, enabling robots to adapt and respond to changing scenarios on their own.



Types of Autonomous Robot Behavior

Robot Type	Description	Example(s) / Applications
Humanoid Robots	Resemble humans in form (head, torso, arms, legs) and sometimes behavior.	Tesla Robot, NVIDIA GR00T
Delivery Robots	Autonomous machines that transport goods (food, parcels, etc.)	Food/package delivery in urban/campus settings
Charging Robots	Robotic arms that autonomously connect and charge electric vehicles using AI vision systems.	Auto EV charging at stations, e.g., Hyundai pilot robot
Construction Robots	Work on job sites, performing tasks like building, material handling, earthmoving, and surveillance.	Automation in construction sites
Research & Education Mobile Robots	Small, extendable robots used for prototyping or teaching robotics concepts; feature similar sensors and software (e.g., ROS) as full-scale robots.	University labs, robotics courses, research prototyping

Data Flow: How Autonomous Behavior Works

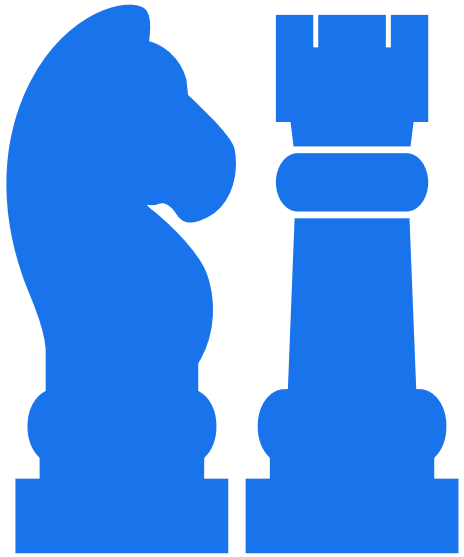




Applications

- Self-Driving Vehicles: Navigation, lane following, dynamic obstacle avoidance.
- Service Robots: Cleaning, delivery, assistance in homes and offices.
- Industrial Automation: Adaptive manufacturing robots, quality inspection bots.
- Search and Rescue: Exploration and mapping of hazardous or unknown areas.
- Healthcare: Autonomous patient-assistance robots.

SLAM (Simultaneous Localization and Mapping)



SLAM is the estimation of the pose of a robot and the map of the environment simultaneously. SLAM is hard because a map is needed for localization and a good pose estimate is needed for mapping

- **Localization**: inferring location given a map.
- **Mapping**: inferring a map given locations.
- **SLAM**: learning a map and locating the robot simultaneously.

SLAM problem is hard because it is kind of a paradox i.e :

- In order to build a map, we need now the position.
- To determine our position, we need a map.

SLAM has multiple parts and each part can be executed in many different ways:



Landmark detection



Data association



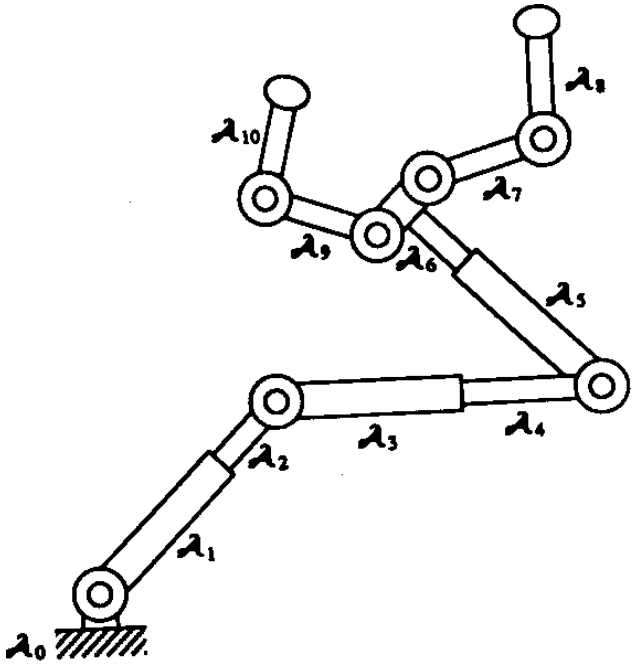
State Estimation

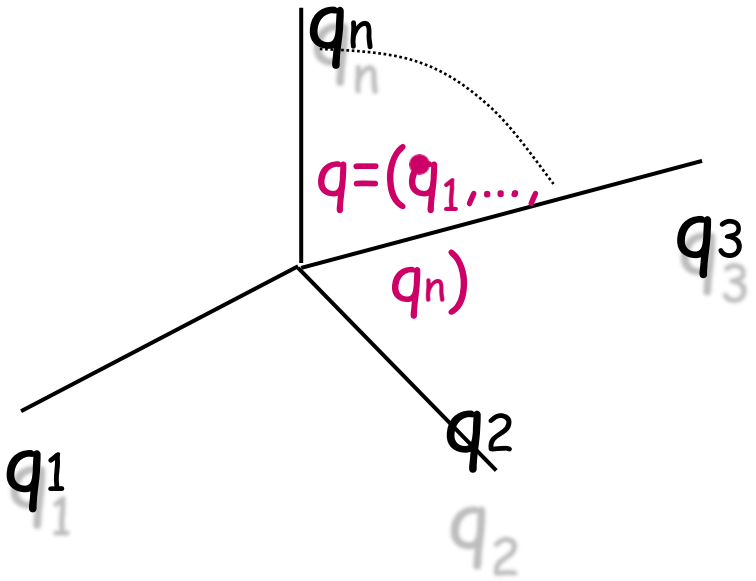


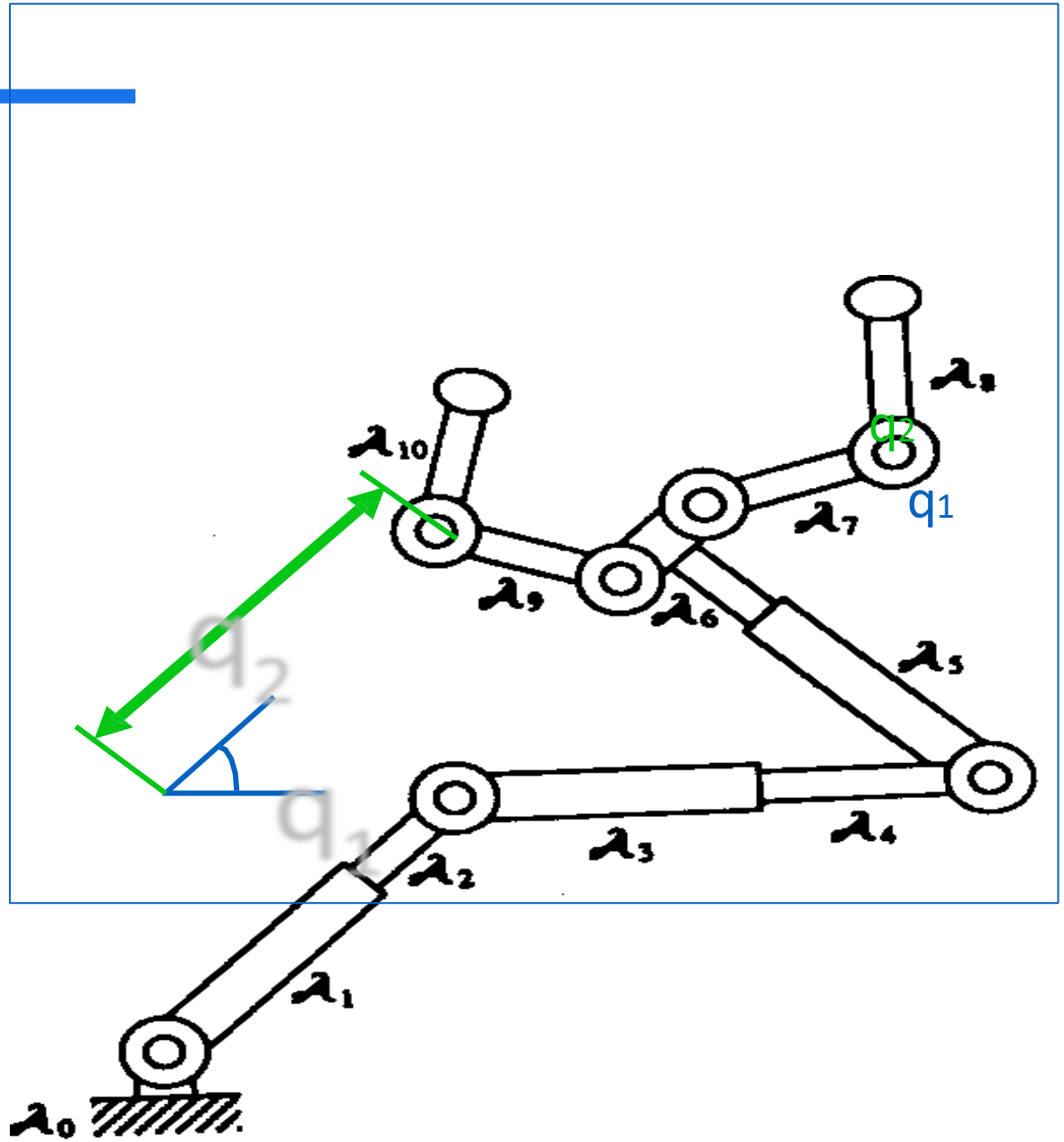
State Update



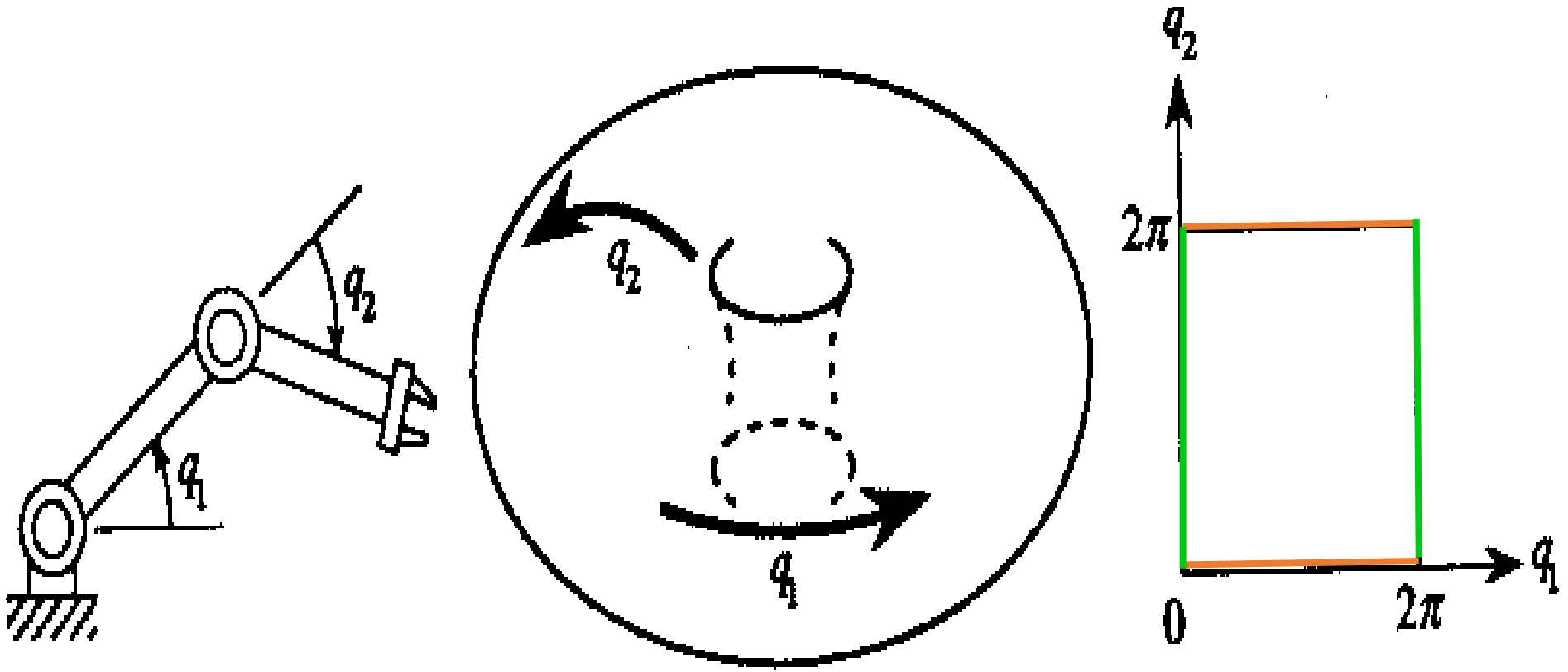
Landmark Update


λ0
λ1
λ2
λ3
λ4
λ5
λ6
λ7
λ8
λ9
λ10


qn
q=(q1,…,
qn)
q3
q1
q2


λ10
λ8
q2
q1
λ7
λ9
λ6
λ5
q2
q1
λ3
λ4
λ2
λ1
λ0

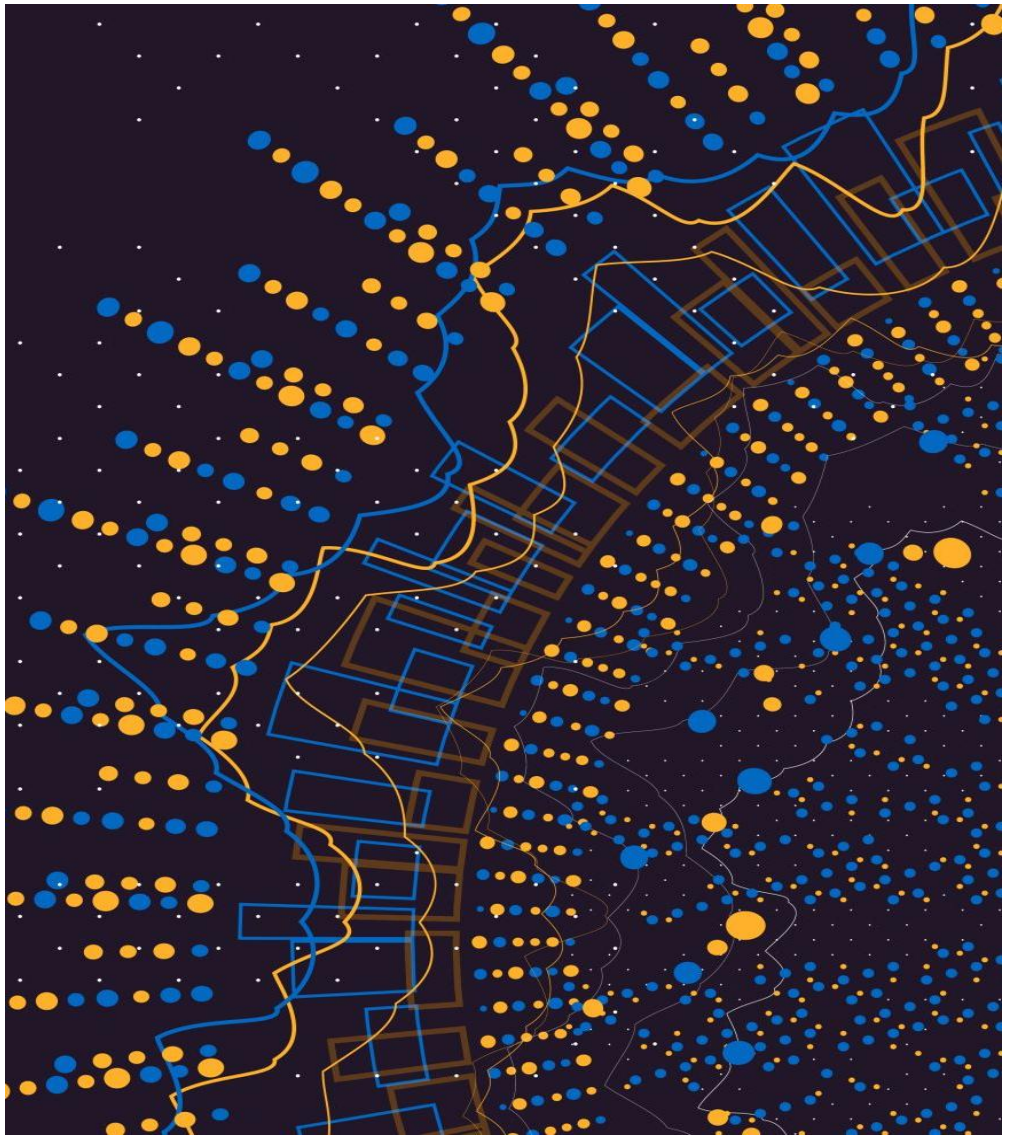
$C = S^1 \times S^1$

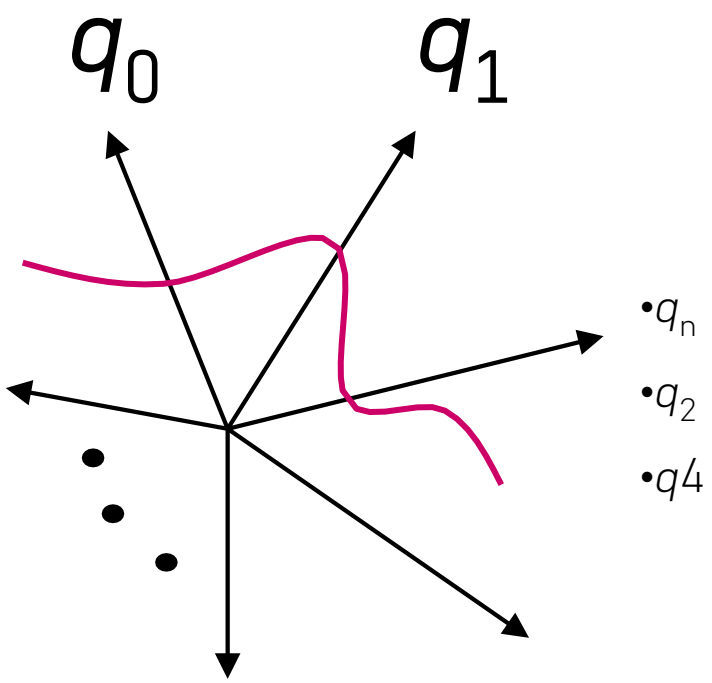


Landmark detection

- SLAM process consists of the following steps:
- In the first step, it uses the environment to update the position of the robot. We can use Odometry but it can be erroneous, we cannot only rely directly on odometry.
- We can use laser scans of the environment to correct the position of the robot. But, it won't work in some environments like underwater.
- Thus, the position of the robot can be better identified by extracting features from the environment.

- **Data association** or data matching is that of matching observed landmarks from different (laser) scans with each other
- **State Estimation:** In this step, we use the odometer data to get the current state estimate.
- **State update:** In this stage, we update our new estimated state by re-observing landmarks.
- **Landmarks update:** In this step, we add new landmarks that are detected in current stage.

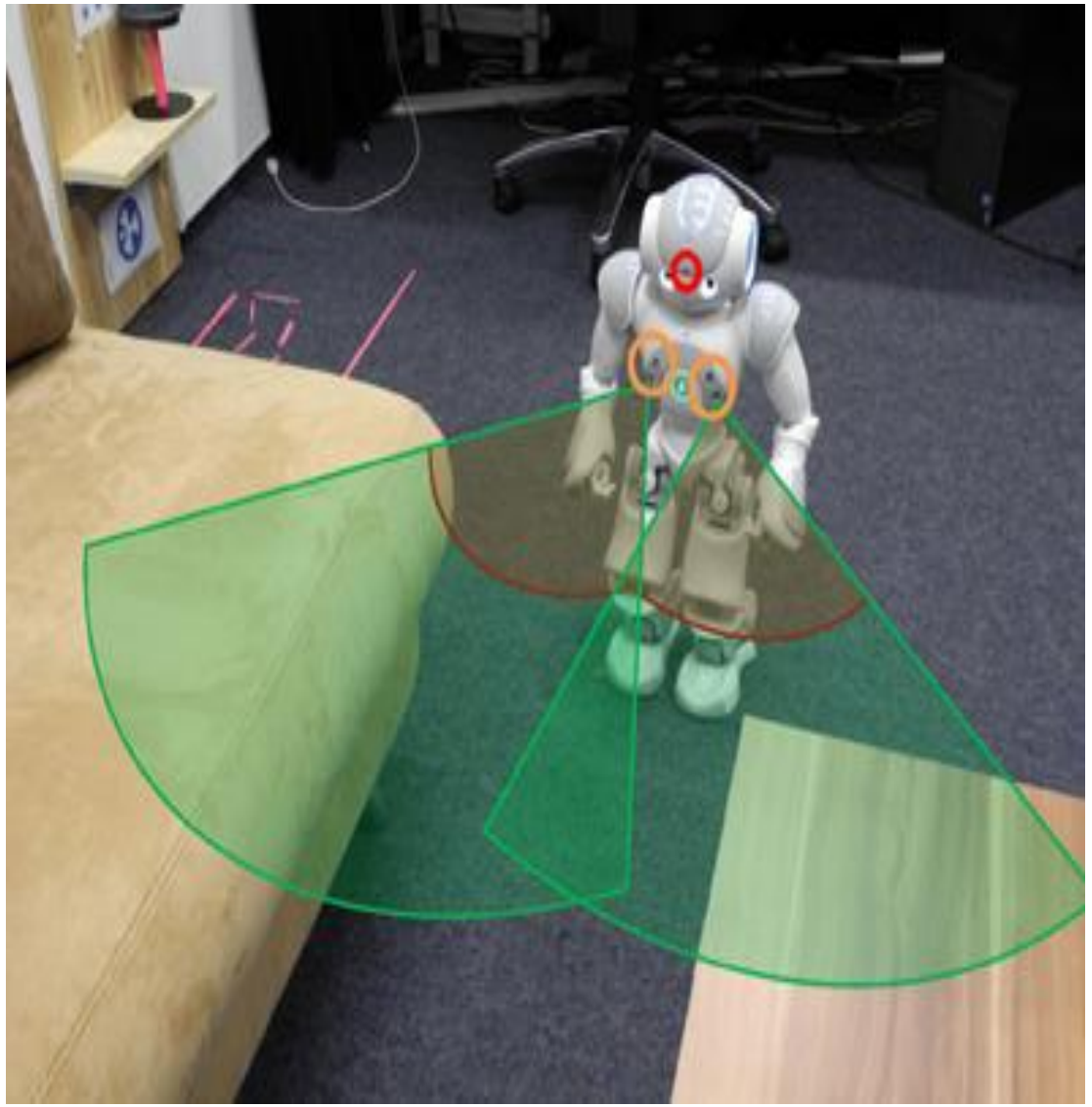


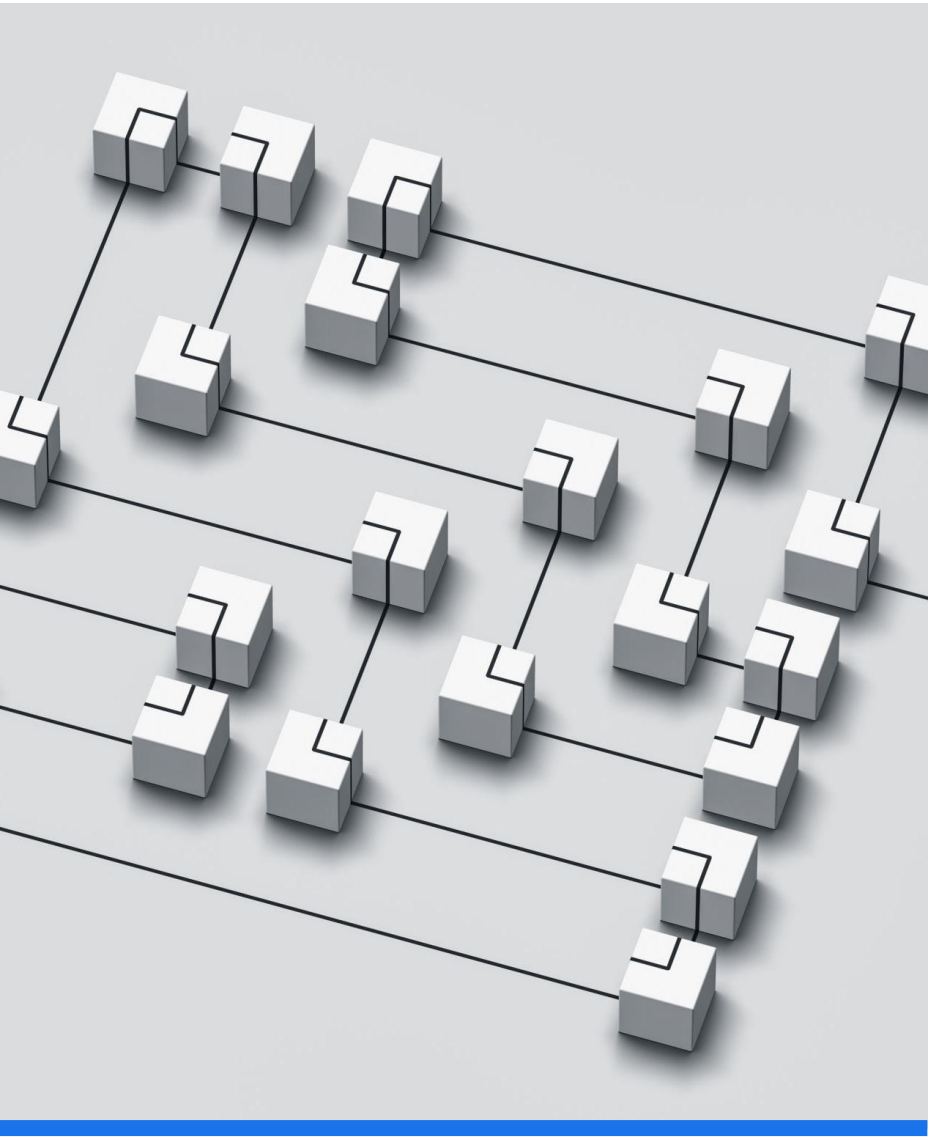

q_0
q_1
•q_n
•q_2
•$q4$
tq(3s)



Navigation in Unknown Environments

- Navigation in unknown environments refers to a robot's ability to move safely and efficiently toward a target without prior maps or information about the surroundings.
- The robot must perceive the environment in real time, make decisions, plan paths, and adapt to unexpected obstacles or changes on its own.





How It Works

1. Perception (Sensing)

- The robot continuously gathers sensory data (from LiDAR, cameras, ultrasonic sensors, etc.) to detect obstacles and free space.

2. Local Mapping & SLAM

- It may use SLAM (Simultaneous Localization and Mapping) to build a map on the fly and localize itself within this new environment.
- Mapping is dynamic, constantly updated as the robot moves and discovers new areas.

3. Path Planning & Obstacle Avoidance

- The robot applies navigation algorithms that work with incomplete, evolving information.

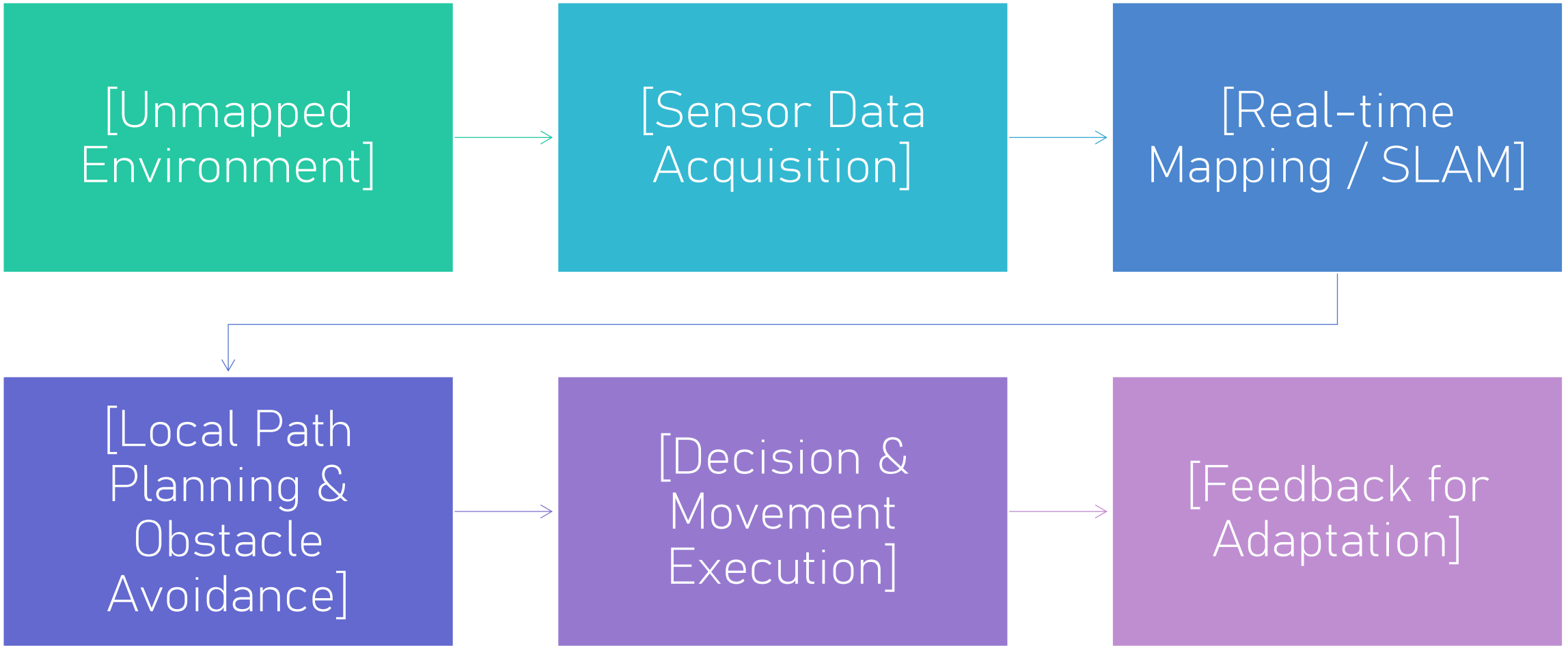
- Two main approaches are:
  - Classical/Heuristic Methods: Use local sensor data to pick short-term safe paths (e.g., potential fields, bug algorithms).
  - AI/Deep Reinforcement Learning (DRL): The robot learns optimal navigation behaviors through trial-and-error and rewards, adapting strategies based on what is encountered.

4. Decision and Movement Execution

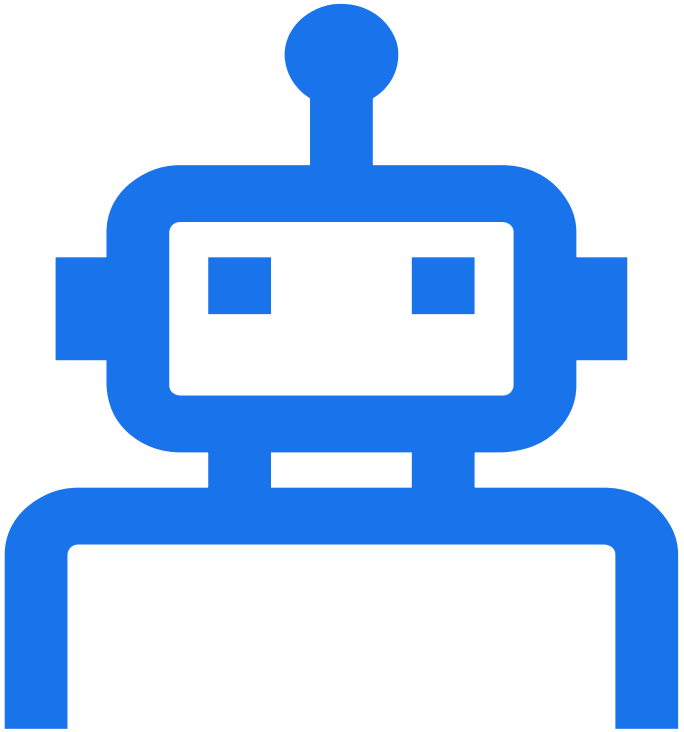
- Plans and trajectory are continuously refined.
- The robot responds in real time to new obstacles (static or moving), rerouting as needed.

5. Feedback Loop

- Successful movement, collisions, or corrections feed back into the perception and planning system, improving future behavior.

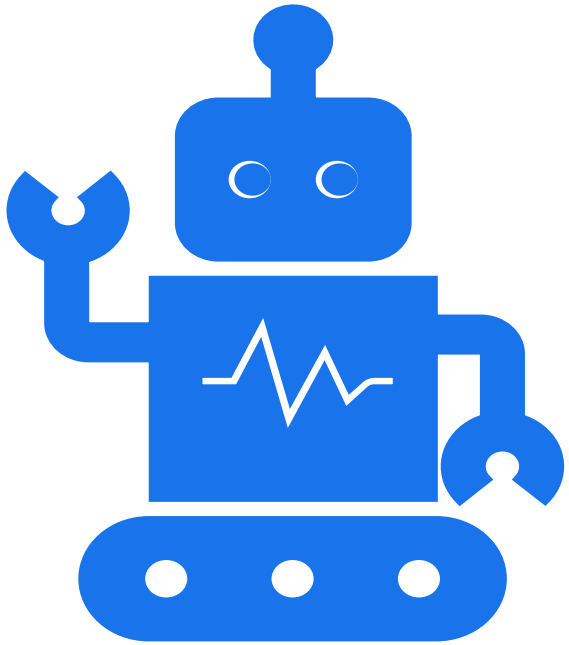

[Unmapped Environment]
[Sensor Data Acquisition]
[Real-time Mapping / SLAM]
[Local Path Planning & Obstacle Avoidance]
[Decision & Movement Execution]
[Feedback for Adaptation]

Applications



- Search and Rescue: Robots navigate collapsed buildings, caves, or disaster zones.
- Warehouse Automation: Adaptive routing in frequently changing layouts.
- Exploration: Drones or rovers operate on planetary surfaces or in underwater expeditions.
- Autonomous Vehicles: Self-driving cars navigate unfamiliar roads or construction zones.

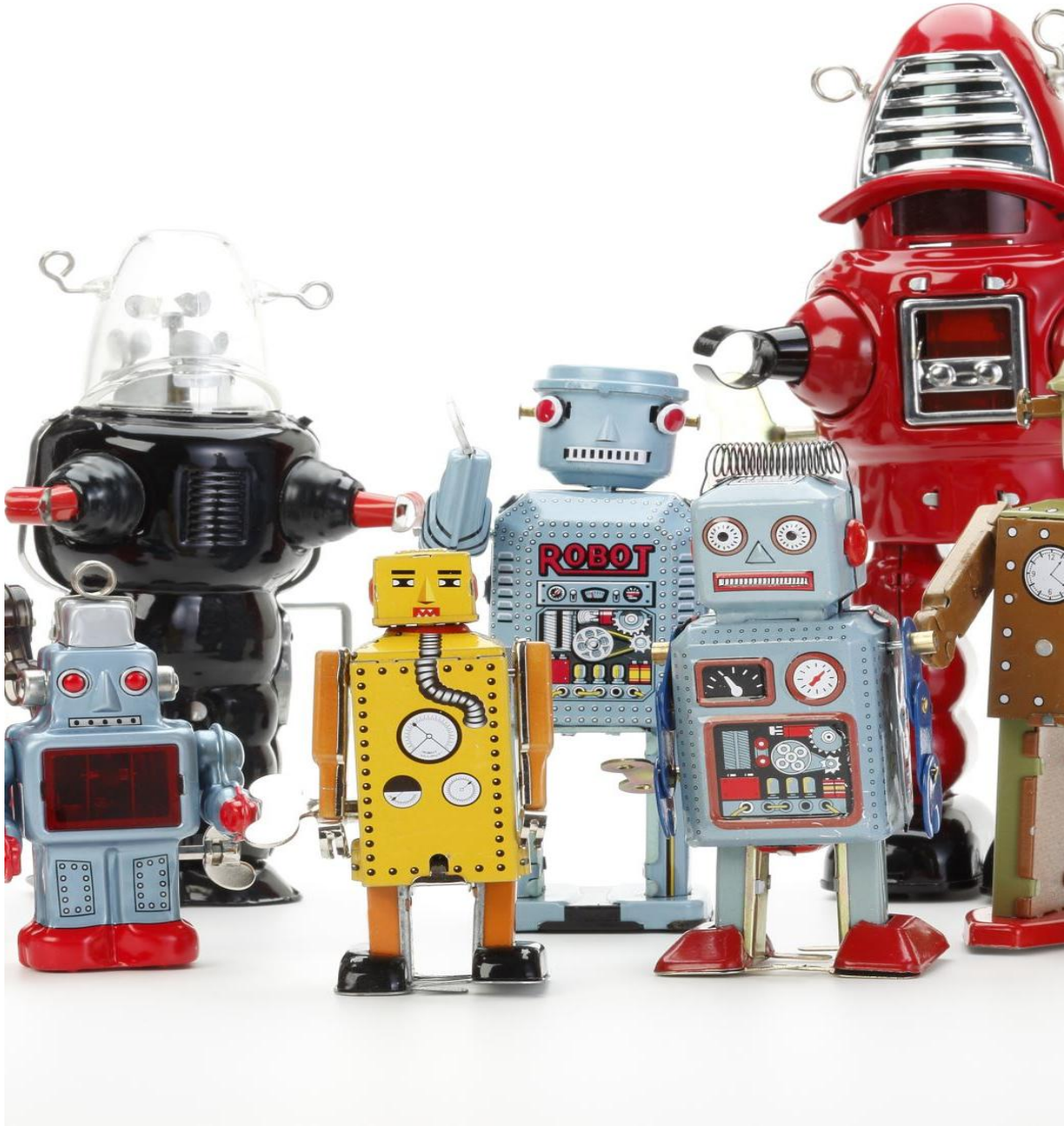
summary:



- Robots use a combination of sensing, real-time mapping (often via SLAM), and adaptive planning (classical or AI-based) to navigate unknown spaces, with continuous feedback enabling them to safely, efficiently, and autonomously accomplish their missions

Multi-robot systems and swarm robotics

- A **multi-robot system** involves a group of robots working together to complete tasks that are difficult, inefficient, or impossible for a single robot.



Key Features:

Coordination – Robots share information and plan together.

Cooperation – Tasks are divided among robots (task allocation).

Communication – Robots often exchange data (directly or indirectly).

Scalability – More robots can be added to improve performance.

Robustness – Failure of one robot doesn't collapse the whole system.

Applications:

- Search and rescue operations
- Exploration (space, deep sea, disaster zones)
- Surveillance and security
- Agricultural robotics (harvesting, monitoring)
- Warehouse automation (e.g., Amazon Kiva robots)

Swarm Robotics

- **Swarm robotics** is a **subset of multi-robot systems**, inspired by **natural swarms** (ants, bees, birds, fish).

• Key Features:

- **Decentralization** – No single leader, decisions made locally.
- **Scalability** – Can involve hundreds or thousands of robots.
- **Emergent behavior** – Complex group behaviors arise from simple rules.
- **Robustness** – Very tolerant to individual robot failures.
- **Bio-inspired algorithms** – Based on stigmergy, flocking, ant colony optimization, etc.

Applications:

- Environmental monitoring (pollution, forest fire detection)
- Search and rescue in dangerous areas
- Military reconnaissance (drone swarms)
- Agriculture (synchronized planting, pest control)
- Planetary exploration (NASA considers swarms for Mars/Moon)

multi-robot systems vs swarm robotics

Aspect	Multi-Robot Systems (MRS)	Swarm Robotics
Control	Can be centralized or decentralized	Fully decentralized
Number of Robots	Usually small to medium (2–50)	Large scale (100+)
Complexity per Robot	Robots may be advanced & specialized	Robots are simple & cheap
Communication	Often explicit and structured	Mostly local, indirect, stigmergy
Inspiration	Engineering-driven	Biology-inspired (ants, bees, etc.)
Example	Warehouse robots cooperating	Drone swarm forming shapes in the sky

Robot programming and applications: Robot Operating System (ROS)

- ROS is **not an actual operating system**; it's a **middleware framework** (a collection of software libraries and tools) that helps developers **build, simulate, and control robots**.

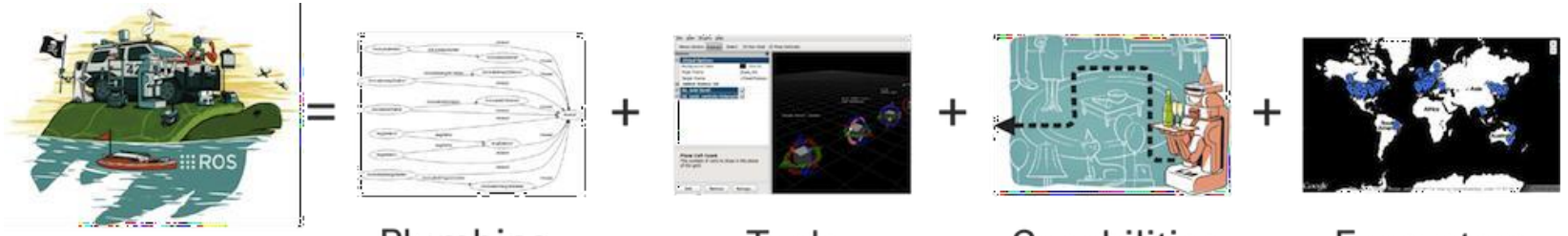
- https://en.wikipedia.org/wiki/Robot_Operating_System#:~:text=Robot%20Operating%20System%20(ROS%20or,code%20and%20embedded%20system%20hardware.

- https://medium.com/software-architecture-foundations/robot-operating-system-2-ros-2-architecture-731ef1867776

ROS

http://www.ros.org

Open Source Robotics Foundation



ROS Box Turtle	March 2, 2010	-	
ROS C Turtle	August 2, 2010	-	
ROS Diamondback	March 2, 2011	-	
ROS Electric Emys	August 30, 2011	-	
ROS Fuerte Turtle	April 23, 2012	-	
ROS Groovy Galapagos	December 31, 2012	July, 2014	
ROS Hydro Medusa	September 4th, 2013	May, 2015	
ROS Indigo Igloo	July 22nd, 2014	April, 2019 (Trusty EOL)	
ROS Jade Turtle	May 23rd, 2015	May, 2017	
ROS Kinetic Kame (Recommended)	May 23rd, 2016	April, 2021 (Xenial EOL)	
ROS Lunar Loggerhead	May 23rd, 2017	May, 2019	

Simulation in ROS

- Several advanced robot simulators can be used with ROS 2, such as Gazebo, Webots, etc. Unlike turtlesim, they provide fairly realistic results relying on physics-based models for robots, sensors, actuators and objects. Hence, what you observe in simulation is very close to what you will get when transferring your ROS 2 controllers to a real robot.

- https://www.openrobotics.org/

How ROS Works

- Nodes: Individual software processes (“nodes”) handle specific tasks (e.g., sensor reading, motion control).
- Topics: Nodes communicate via asynchronous message passing (“topics”) using a publish-subscribe model.
- Services: Synchronous request-response communication for defined tasks.
- Hardware Abstraction: ROS decouples hardware details, allowing you to develop software for different robot platforms without adjusting low-level drivers.
- Packages: Modular software components (libraries, algorithms) that can be reused and integrated into various robotics projects.
- Master & Graph Architecture: The ROS Master manages registration and communication between nodes.

Applications of ROS

Area	Description
Industrial Automation	Controls robotic arms, AGVs, complex automation sequences .
Healthcare Robotics	Enables surgical robots, assistive devices, and health monitoring robots .
Autonomous Vehicles	Supports sensing, localization, mapping, planning, and control for cars/drones .
Agricultural Robotics	Manages planting, harvesting, and crop monitoring systems .
Education/Research	Used worldwide as the standard for teaching, prototyping, and simulation

Benefits of ROS

- Open Source & Community Driven: Free with worldwide, collaborative support and extensive resources.
- Hardware Abstraction: Code easily transferred between simulation and real robots.
- Modularity & Reusability: Develop, test, and integrate diverse software packages with ease.
- Scalability: Works for single robots or highly distributed multi-robot systems.
- Powerful Developer Tools: Includes visualization, debugging, and monitoring tools for efficient development.
- Integration with AI & ML: Enables modern approaches for perception, planning, and adaptive behavio